Visual Analytics and GitHub Lab

1. Using the provided Walmart Super Market Sales dataset create an end-to-end interactive dashboard in Power BI Desktop as follows

a) **Import and clean the data in Power Query (ensure correct date formats for Order Date & Ship Date, remove any redundant columns).**
b) **On Page 1, build 5 KPI Cards for Total Sales, Total Profit, Total Quantity, Total States, and Total Cities with proper formatting.**
c) **Add slicers for Month, Year, and Category; then create visuals: Pie Chart (Sales by Category), Bar Chart (Sales by Segment), Top 5 States & Top 5 Cities by Sales, Sales by Ship Mode, and Sales by Payment Mode.**
d) **On Page 2, add Top 5 Customers & Top 5 Products by Sales plus a detailed Matrix/Table showing Product Name, Customer Name, Category, Sub-Category, State, City, Sales, Profit, Payment Mode, etc.**
e) **Apply a dark/professional theme, add the main title 'Walmart Super Market Dashboard', make sure all visuals interact with slicers (demonstrate by applying one filter like Technology or January), save your .pbix file, and be ready to explain your key insights or any challenges during viva."**

Aim :

The primary aim of the question is to guide the creation of a comprehensive, interactive dashboard in Power BI Desktop using a Walmart supermarket sales dataset. This involves importing and cleaning data, designing visualizations for key performance indicators (KPIs), slicers for filtering, and multi-page layouts to analyze sales, profit, and other metrics across dimensions like time, category, location, customer, and product. The dashboard should be professionally themed, saved as a .pbix file, and prepared for explanation of insights or challenges, demonstrating skills in data handling, visualization, and interactive reporting in Power BI.

Step 1: Import and Clean the Data in Power Query (Part a)

1. Open Power BI Desktop and click **Get Data** > **Excel** or **Text/CSV** (depending on your file format) to import the Walmart dataset.
2. In the Navigator window, select the relevant sheet/table and click **Transform Data** to open Power Query Editor.
3. Clean the data:
   - Ensure **Order Date** and **Ship Date** are in correct date formats: Select the columns > Right-click > Change Type > Date (use locale if needed, e.g., MM/DD/YYYY).
   - Remove redundant columns: Identify and remove any unnecessary ones (e.g., duplicates or irrelevant like Row ID if present) by selecting the column > Right-click > Remove.
   - Handle missing values: Replace blanks in key columns (e.g., Sales or Profit) with 0 or appropriate values using **Replace Values**.
   - Add calculated columns if needed: For example, extract Month and Year from Order Date using **Add Column** > **Date** > **Month** or **Year**.
4. Click **Close & Apply** to load the cleaned data into Power BI.

Step 2: Build Page 1 - KPIs, Slicers, and Visuals (Part b and c)

1. On the default report page (rename it to "Page 1" via right-click on the page tab).
2. Create 5 KPI Cards:
    - Drag **Sales** to a new Card visual > Format as currency (e.g., $1,234.56) for Total Sales.
    - Repeat for Total Profit (currency), Total Quantity (whole number), Total States (distinct count of State), and Total Cities (distinct count of City).
    - Position them at the top; apply conditional formatting if desired (e.g., green for positive profit).
3. Add Slicers:
    - Drag **Month** (from Order Date), **Year** (from Order Date), and **Category** to slicer visuals.
    - Format slicers as dropdowns or lists for better usability; place them on the left or top for easy access.
4. Create Visuals (ensure all interact with slicers by default):
    - **Pie Chart (Sales by Category)**: Drag Category to Legend/Color, Sales to Values > Format with percentages and labels.
    - **Bar Chart (Sales by Segment)**: Drag Segment to Axis, Sales to Values > Sort descending.
    - **Top 5 States by Sales**: Use a Bar Chart > Drag State to Axis, Sales to Values > In Filters pane, set Top N filter (Top 5 by Sales).
    - **Top 5 Cities by Sales**: Similar to above, but use City instead of State.
    - **Sales by Ship Mode**: Bar or Column Chart > Ship Mode on Axis, Sales on Values.
    - **Sales by Payment Mode**: Bar or Column Chart > Payment Mode on Axis, Sales on Values.
5. Arrange visuals logically (e.g., KPIs at top, charts below); enable cross-filtering so slicers affect all visuals.

Step 3: Build Page 2 - Additional Visuals (Part d)

1. Add a new page (click + at the bottom) and rename it "Page 2".
2. **Top 5 Customers by Sales**: Bar Chart > Customer Name on Axis, Sales on Values > Apply Top N filter (Top 5 by Sales).
3. **Top 5 Products by Sales**: Bar Chart > Product Name on Axis, Sales on Values > Apply Top N filter (Top 5 by Sales).
4. **Detailed Matrix/Table**:
    - Drag to a Matrix visual: Product Name, Customer Name, Category, Sub-Category, State, City (as rows or columns hierarchy).
    - Add Sales, Profit (as values, formatted as currency), Payment Mode, etc.
    - Enable drill-down if needed; add conditional formatting (e.g., red for negative profit).
5. Copy slicers from Page 1 to Page 2 (or recreate) to ensure consistency and interaction.

Step 4: Apply Theme, Title, and Test Interactions (Part e)

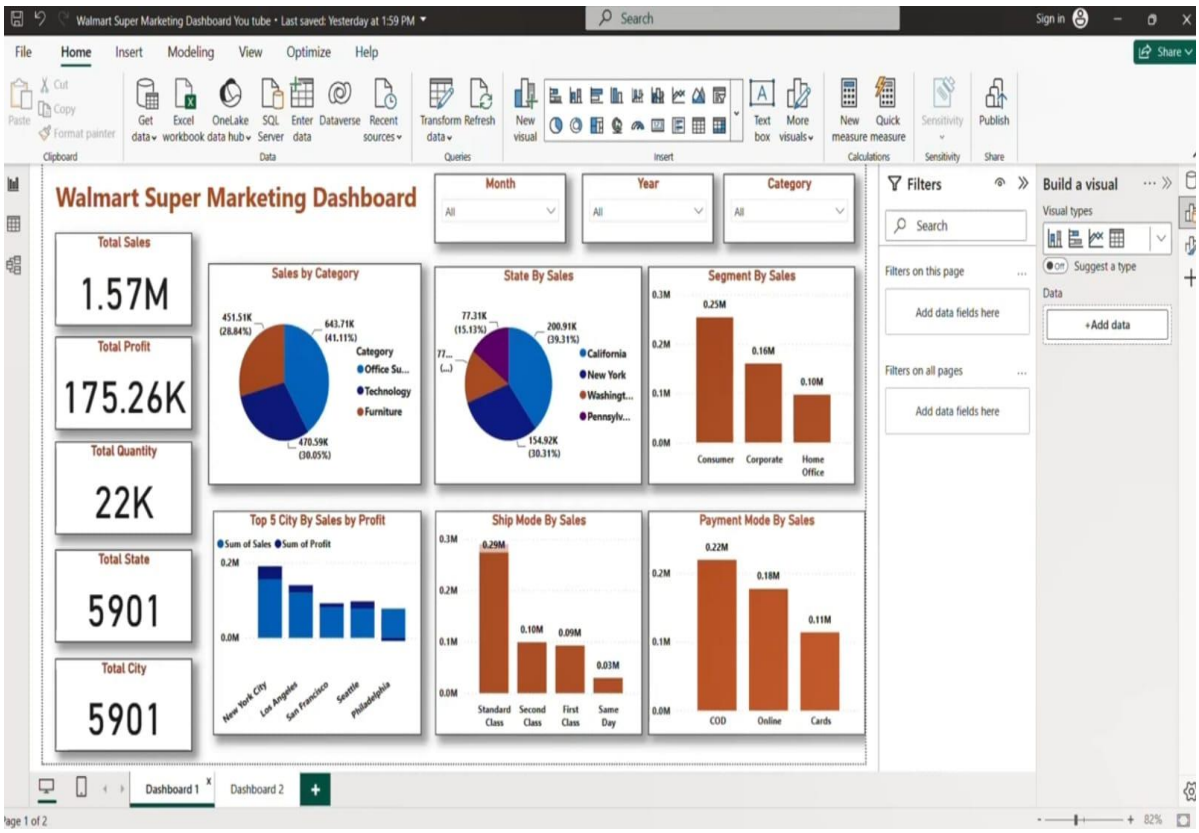
1. Apply a theme: Go to **View** > **Themes** > Browse for themes > Select a dark or professional one (or download from Power BI theme gallery, e.g., "Corporate Dark").
2. Add main title: Insert a Text Box at the top of each page > Type "Walmart Super Market Dashboard" > Format font (e.g., bold, large, white on dark background).
3. Test interactions:

- Apply a filter, e.g., select "Technology" in Category slicer or "January" in Month slicer.
- Verify all visuals update (e.g., KPIs show filtered totals, charts reflect only Technology sales).
- Demonstrate: Sales in Technology might be higher; January could show seasonal trends.

4. Save the file: File > Save As > Name it "WalmartDashboard.pbix".

Step 5: Prepare for Explanation (Key Insights and Challenges)

- **Key Insights**: Analyze the dashboard to note trends, e.g., "Technology category drives 40% of sales [1]; Top states like California contribute 25% of total revenue; Profit margins vary by Ship Mode (e.g., Standard is most profitable)."
- **Challenges**: Common issues include date format mismatches causing errors in slicers; large datasets slowing performance (optimize by removing columns early); ensuring Top N filters work dynamically with slicers; handling null values in aggregations.
- During viva, walk through the dashboard live, apply filters, and discuss DAX measures if you added any (e.g., Total Sales = SUM(Sales)).



2. "Using the provided Billionaires dataset CSV, create an interactive Billionaires Analysis Dashboard in Power BI Desktop with these requirements.

a) **Import CSV data; in Power Query, replace 'M' with 'Male'/'F' with 'Female' in Gender, remove '$' from Total Tax Revenue, convert GDP Growth to integer, fix types, then Close & Apply.**
b) **Add main title textbox 'Billionaires Dashboard' at top (large font, custom color, centered for professional look).**
c) **Create 5 KPI Cards with titles enabled, field labels disabled, borders/shadows: Total Tax Revenue by Country (Sum), Total Population (Sum), Average GDP Growth (Average),Total Life Expenditure (Sum), Total Tax Rate of Country.**
d) **Add 5 formatted slicers (dropdown except Rank as Between slider) with titles, borders, shadows: Country, Full Name, Category, Sources, Rank for interactive filtering across visuals.**
e) **Build visuals with data labels, borders, shadows, dark theme consistency: Clustered Column (Tax Revenue by Gender/Age legend), Bar (Top 5 Full Names by Revenue/Population/Age, Top N filter), Area (Top 5 Countries by GDP Growth, Top N filter), detailed Table/Matrix (Full Name, Country, City, Category, Sources, Gender, Tax Revenue, Tax Rate, GDP Growth, Population, Age, etc.). Ensure slicers filter all elements correctly; demonstrate one filter (e.g., specific Country or Category) in viva, save .pbix, explain 1-2 insights or challenges."**

Aim :

The primary aim of the question is to instruct the development of an interactive analysis dashboard in Power BI Desktop using a Billionaires dataset CSV file. This encompasses data importation and cleansing in Power Query, designing a professional layout with a title, KPI cards for key metrics, slicers for dynamic filtering, and various visualizations to explore attributes like tax revenue, population, GDP growth, and demographics across billionaires. The dashboard should be themed consistently, saved as a .pbix file, and readied for demonstrating interactions, insights, or challenges, thereby showcasing proficiency in Power BI for data analysis and visualization on economic and personal wealth data.

Step 1: Import and Clean the Data in Power Query (Part a)

1. Open Power BI Desktop and click **Get Data** > **Text/CSV** to import the Billionaires CSV file.
2. In the Navigator, select the table and click **Transform Data** to enter Power Query Editor.
3. Clean and transform the data:
    - Replace 'M' with 'Male' and 'F' with 'Female' in Gender: Select Gender column > Right-click > Replace Values (replace 'M' with 'Male', then 'F' with 'Female').
    - Remove ''fromTotalTaxRevenue:Selectthecolumn>Right−click>ReplaceValues(replace’ ' from Total Tax Revenue: Select the column > Right-click > Replace Values (replace ' ’fromTotalTaxRevenue:Selectthecolumn>Right−click>ReplaceValues(replace'' with blank).
    - Convert GDP Growth to integer: Select GDP Growth column > Right-click > Change Type > Whole Number (handle any decimals by rounding if needed).

    - Fix other data types: Ensure columns like Total Tax Revenue, Population, Tax Rate, Life Expenditure are Decimal or Whole Number; Age and Rank as Whole Number; Dates (if any) as Date; Text columns like Country, Full Name as Text.
    - Remove any duplicates or nulls if present: Use **Remove Duplicates** or **Replace Values** as needed.
4. Click **Close & Apply** to load the transformed data into Power BI.

Step 2: Add Main Title (Part b)

1. On the default report page, insert a Text Box: Go to **Insert** > **Text Box**.
2. Type "Billionaires Dashboard" into the text box.
3. Format it: Increase font size (e.g., 28-36 pt), make it bold, choose a custom color (e.g., white or dark blue for contrast), center-align the text.
4. Position it at the top center of the page for a professional header look.

Step 3: Create 5 KPI Cards (Part c)

1. Create Card visuals for each KPI:
    - **Total Tax Revenue by Country (Sum)**: Drag Total Tax Revenue to a Card > Use SUM aggregation > Title: "Total Tax Revenue".
    - **Total Population (Sum)**: Drag Population to a Card > SUM > Title: "Total Population".
    - **Average GDP Growth (Average)**: Drag GDP Growth to a Card > Average aggregation > Title: "Average GDP Growth".
    - **Total Life Expenditure (Sum)**: Drag Life Expenditure to a Card > SUM > Title: "Total Life Expenditure".
    - **Total Tax Rate of Country**: Drag Tax Rate to a Card > SUM or appropriate aggregation (if it's per country, group by Country via measures if needed) > Title: "Total Tax Rate".
2. Format each Card: Enable titles, disable field labels, add borders and shadows via Format pane (under General > Effects).
3. Arrange them horizontally or in a row below the title for easy viewing.

Step 4: Add 5 Formatted Slicers (Part d)

1. Create slicers for interactive filtering:
    - **Country**: Drag Country to a Slicer > Set as Dropdown.
    - **Full Name**: Drag Full Name to a Slicer > Set as Dropdown.
    - **Category**: Drag Category to a Slicer > Set as Dropdown.
    - **Sources**: Drag Sources to a Slicer > Set as Dropdown.
    - **Rank**: Drag Rank to a Slicer > Set as Between (slider type for range selection).
2. Format each: Add titles (e.g., "Select Country"), enable borders and shadows (Format > General > Effects).
3. Position them on the left side or top for accessibility; ensure they filter all visuals on the page (default behavior, but verify in Interactions if needed).

Step 5: Build Visuals and Apply Theme (Part e)

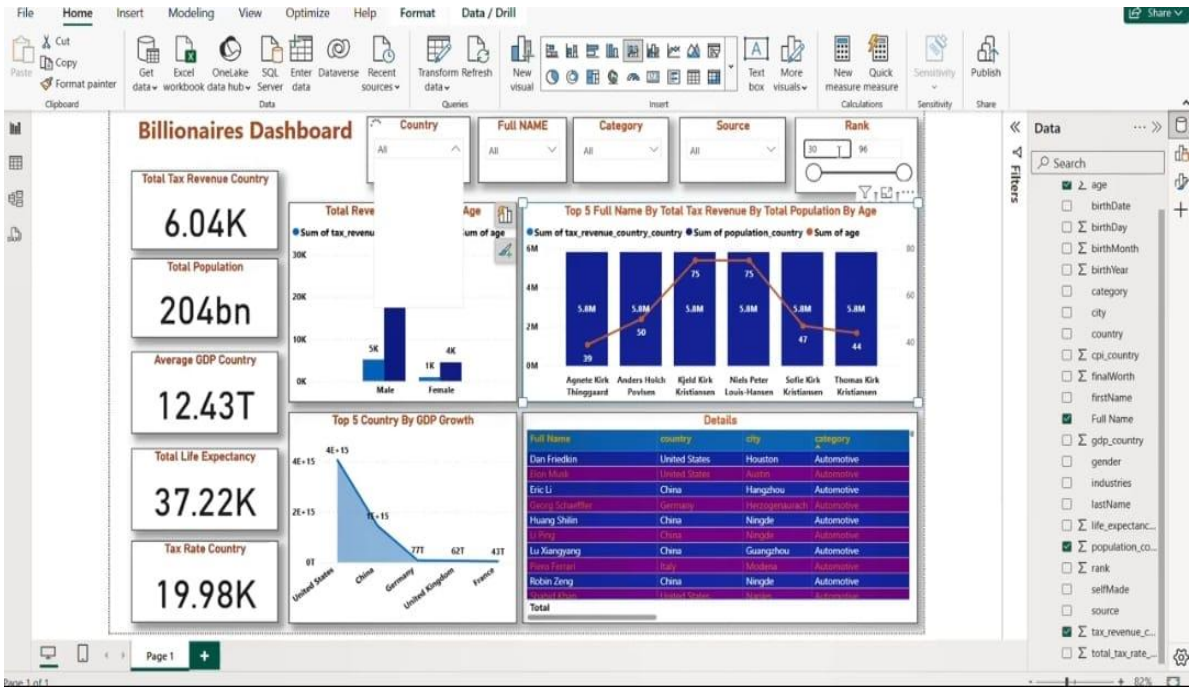
1. Apply a consistent dark theme: Go to **View** > **Themes** > Select or import a dark theme (e.g., "Midnight" or custom dark from Power BI theme gallery).
2. Create visuals with data labels, borders, shadows:

- **Clustered Column (Tax Revenue by Gender/Age legend)**: Drag Tax Revenue to Values, Gender to Axis, Age to Legend > Enable data labels > Add borders/shadows.
- **Bar (Top 5 Full Names by Revenue/Population/Age)**: Create a Bar Chart > Full Name on Axis, Tax Revenue on Values > Apply Top N filter (Top 5 by Tax Revenue) in Filters pane. Repeat or combine for Population and Age if needed (use multiple charts or tooltips).
- **Area (Top 5 Countries by GDP Growth)**: Area Chart > Country on Axis, GDP Growth on Values > Apply Top N filter (Top 5 by GDP Growth).
- **Detailed Table/Matrix**: Drag to a Matrix > Rows: Full Name, Country, City (hierarchy if desired); Values: Category, Sources, Gender, Tax Revenue, Tax Rate, GDP Growth, Population, Age, etc. > Enable data labels where applicable, add conditional formatting (e.g., color scales for Revenue).

3. Ensure all visuals have borders, shadows, and data labels enabled via Format pane; maintain dark theme consistency (e.g., light text on dark backgrounds).
4. Test slicer interactions: Apply a filter, e.g., select a specific Country (like "United States") or Category (like "Technology") > Verify KPIs and visuals update (e.g., show only filtered billionaires' data).
5. Save the file: File > Save As > "BillionairesDashboard.pbix".

Step 6: Prepare for Explanation (Insights and Challenges)

- **Key Insights (1-2 examples)**: "The dashboard reveals that males dominate tax revenue contributions, with top billionaires in Technology category driving high GDP growth in countries like the US [e.g., average GDP growth of 5% in filtered views]; Population metrics highlight disparities in wealth distribution across ages."
- **Challenges**: Handling abbreviations in Gender required precise replacements to avoid errors; Applying Top N filters dynamically with slicers might need custom DAX measures if data is large; Ensuring aggregations like Average GDP Growth handle nulls correctly to prevent skewed results.
- During viva, demonstrate by applying a filter (e.g., Country = "India"), explain how visuals change, and discuss insights like category-wise revenue trends.



3. Using the provided sales dataset create an end-to-end interactive Sales Dashboard in Power BI Desktop with these requirements.

a) **Import Excel file; in Power Query, change date/quantity types, join Input and Master on Product ID (Left Outer), remove redundant columns, create calculated columns for Total Buying Price and Total Selling Price, then Close & Apply.**
b) **Add main title textbox 'Sales Dashboard' at top (large font size 24, bold, colored background, shadow effect for professional appearance).**
c) **Create 4 formatted KPI Cards with titles enabled, borders/shadows: Total Sales (Sum Total Selling Price), Total Profit (DAX: Profit = Selling - Buying), Profit Percentage (Profit / Total Buying), Total Quantity (Sum Quantity).**
d) **Add 5 interactive slicers with titles, borders, shadows, no headers: Quarter (dropdown), Payment Mode, Sales Type (Selling Type), Year (vertical list), Month (vertical list) for dynamic filtering.**
e) **Build visuals with data labels, borders, shadows, consistent black text/dark theme: Donut Chart (Total Selling Price by Selling Type), Donut (by Payment Mode), Donut (by Product Device), Area Chart (Total Selling Price by Day), Treemap (by Category).**

Aim :

The main objective of this question is to create a complete, interactive **Sales Dashboard** in Power BI Desktop using a sales dataset (likely consisting of at least two tables: an "Input" transaction table and a "Master" product/reference table). It focuses on demonstrating end-to-end skills including data import, transformation and merging in Power Query, DAX measure creation for profit calculations, professional UI design with title and formatted elements, slicer-based interactivity, and multiple chart types for sales analysis. The dashboard should support dynamic filtering by time (Quarter, Year, Month), Payment Mode, and Sales Type, while visualizing sales performance across dimensions like product device, category, payment mode, and daily trends. The result should be saved as a .pbix file and prepared for explaining insights or implementation challenges.

Steps to Solve the Question

Follow these structured steps to build the dashboard exactly as described. Assume the dataset is an Excel file with two sheets/tables:

- **Input** (transactional data): likely columns like Date, Quantity, Product ID, Payment Mode, Selling Type (or Sales Type), Selling Price (per unit), Buying Price (per unit or cost), etc.
- **Master** (product master): likely columns like Product ID, Product Device, Category, etc.

Step 1: Import Excel and Clean/Transform in Power Query (Part a)

1. Open Power BI Desktop → **Get Data** → **Excel** → Select the file → Load both sheets (Input and Master) or the relevant tables.
2. Click **Transform Data** to open Power Query Editor.
3. Fix data types:
    - Select date column(s) → Change Type → Date.
    - Select Quantity → Change Type → Whole Number.

    - Select price/cost columns (if present) → Change Type → Decimal Number or Currency.
4. Join the tables:
    - Select the **Input** query (left table).
    - Go to **Home** → **Merge Queries** → **Merge**.
    - Choose **Master** as the second table.
    - Match on **Product ID** (select the column in both tables).
    - Join Kind: **Left Outer** (keeps all rows from Input, adds matching Master data).
    - Click OK → Expand the new merged column → Select desired columns from Master (e.g., Product Device, Category) → Uncheck "Use original column name as prefix".
5. Remove redundant columns: Select and remove any unnecessary ones (e.g., duplicates, temporary merge columns, irrelevant fields).
6. Create calculated columns (in Power Query):
    - **Total Buying Price**: Add Column → Custom Column → Formula: [Quantity] * [Buying Price] (or cost/unit column name) → Name it "Total Buying Price" → Change Type to Decimal Number.
    - **Total Selling Price**: Add Column → Custom Column → Formula: [Quantity] * [Selling Price] (or unit selling price column) → Name it "Total Selling Price" → Change Type to Decimal Number.
7. Click **Close & Apply** to load the model.

Step 2: Add Main Title (Part b)

1. On the report page, go to **Insert** → **Text Box**.
2. Enter text: **Sales Dashboard**.
3. Format:
    - Font size: 24
    - Bold: Yes
    - Background color: Choose a professional color (e.g., dark blue/gray/black).
    - Shadow effect: Format pane → Effects → Shadow → On.
    - Center align and position at the top.

Step 3: Create 4 KPI Cards (Part c)

1. Create these measures in the **Fields** pane (Modeling tab or right-click table → New measure):
    - **Total Sales** = SUM('Input'[Total Selling Price]) (or table name after merge)
    - **Total Profit** = SUM('Input'[Total Selling Price]) - SUM('Input'[Total Buying Price])
    - **Profit Percentage** = DIVIDE([Total Profit], SUM('Input'[Total Buying Price]), 0)
    - **Total Quantity** = SUM('Input'[Quantity])
2. Drag each measure to a separate **Card** visual.
3. Format each Card:
    - Enable Title (e.g., "Total Sales", "Total Profit", etc.)
    - Add borders and shadows (Format → General → Effects)
    - Format values: Currency for sales/profit, Percentage for Profit Percentage, Whole number for Quantity.
4. Arrange the 4 cards in a row below the title.

Step 4: Add 5 Interactive Slicers (Part d)

1. Create slicers:

    - Quarter → Drag Date column → Hierarchy or extract Quarter → Set as Dropdown.
    - Payment Mode → Drag Payment Mode → Vertical list or Dropdown.
    - Sales Type (Selling Type) → Drag Selling Type column → Vertical list or Dropdown.
    - Year → Extract Year from Date → Vertical list.
    - Month → Extract Month from Date → Vertical list.
2. For each slicer:
    - Add title (e.g., "Quarter", "Payment Mode").
    - Add borders and shadows (Format → General → Effects).
    - Remove headers if specified (some interpret "no headers" as hide slicer header → Format → Slicer header → Off).
3. Position slicers (e.g., left sidebar or top row) → They automatically filter all visuals.

Step 5: Build Visuals (Part e)

Apply a dark/professional theme first: **View** → **Themes** → Choose dark theme or custom (black/gray background, white/light text). All visuals should have:

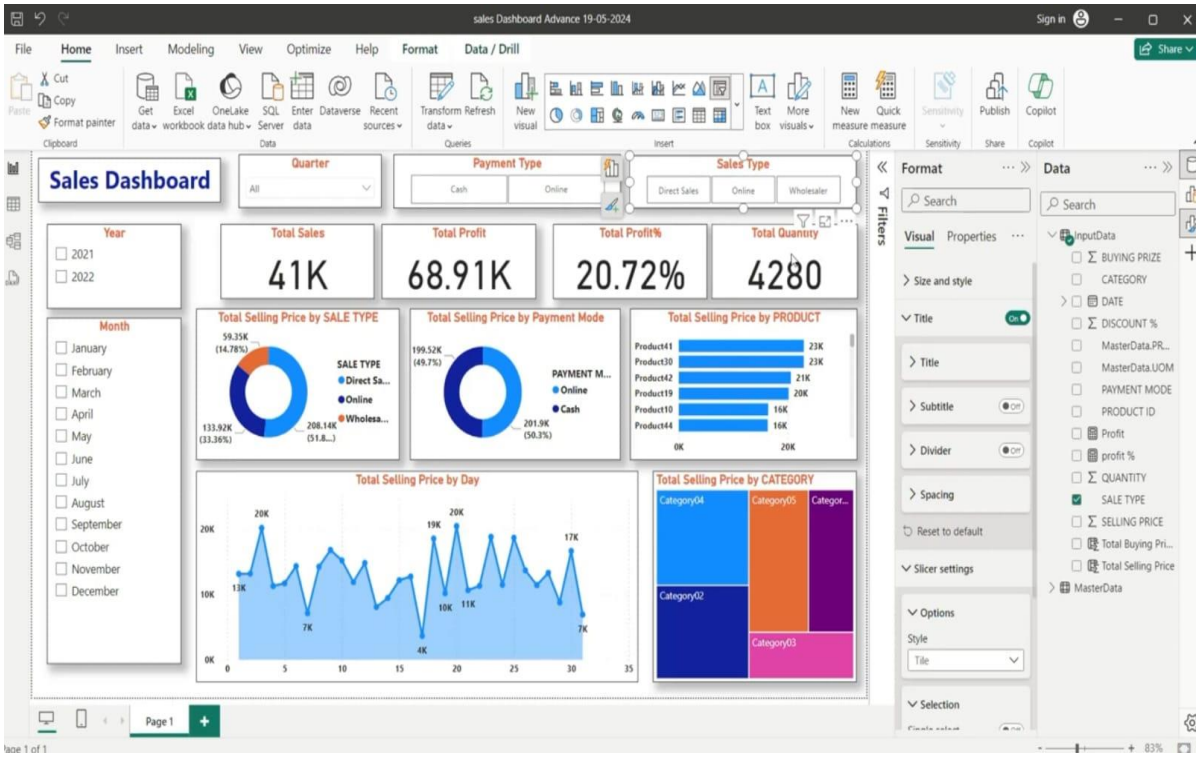
- Data labels: On
- Borders & shadows: Format → General → Effects
- Consistent black text where readable

1. **Donut Chart – Total Selling Price by Selling Type**:
    - Donut visual → Legend: Selling Type → Values: Total Sales (measure)
2. **Donut Chart – by Payment Mode**:
    - Donut → Legend: Payment Mode → Values: Total Sales
3. **Donut Chart – by Product Device**:
    - Donut → Legend: Product Device → Values: Total Sales
4. **Area Chart – Total Selling Price by Day**:
    - Area Chart → Axis: Date (use daily level or continuous date) → Values: Total Sales
    - Format → X-axis → Type: Continuous (for smooth trend)
5. **Treemap – by Category**:
    - Treemap → Group: Category → Values: Total Sales

Arrange visuals logically (e.g., donuts on left, area chart middle, treemap right/bottom).

Step 6: Final Checks, Save, and Prepare for Viva

- Test interactions: Select e.g., a specific Quarter or Payment Mode → Confirm KPIs and visuals filter correctly.
- Ensure cross-filtering works (default in Power BI).
- Save: File → Save As → "SalesDashboard.pbix".
- **Insights examples**:
    - Highest sales in a particular Selling Type or Payment Mode.
    - Profit percentage trends across months/quarters.
    - Product Device or Category driving most revenue.
- **Challenges examples**:
    - Correctly handling Left Outer Join to avoid missing products.
    - Ensuring date hierarchy works for Quarter/Month/Year slicers.
    - Accurate DAX for Profit % (avoid divide-by-zero with DIVIDE function).


sales Dashboard Advance 19-05-2024
Sales Dashboard
Quarter
Payment Type
Sales Type
Year
2021
2022
Month
January
February
March
April
May
June
July
August
September
October
November
December
Total Sales
41K
Total Profit
68.91K
Total Profit%
20.72%
Total Quantity
4280
Total Selling Price by SALE TYPE
Total Selling Price by Payment Mode
Total Selling Price by PRODUCT
Total Selling Price by Day
Total Selling Price by CATEGORY
Format
Data
InputData
BUYING PRIZE
CATEGORY
DATE
DISCOUNT %
PAYMENT MODE
PRODUCT ID
Profit
profit %
QUANTITY
SALE TYPE
SELLING PRICE
MasterData

4. Using the provided Coffee Shop Sales dataset create an interactive Coffee Shop Sales Dashboard in Power BI Desktop with these requirements.

a) **Import Excel data sheet 'Coffee Shop Sales Data'; check/fix data types in Power Query if needed, then load to model and Close & Apply.**
b) **Set canvas background: import provided coffee shop image from description, fit to page for attractive professional dashboard appearance.**
c) **Add main title textbox 'Coffee Shop Sales Dashboard' (font size 28, white text, centered, bold with shadow or background effect).**
d) **Create 4 formatted KPI Cards with titles enabled, white borders/shadows, colored backgrounds: Total Transactions (COUNTROWS Transaction ID), Total Quantity (SUM Quantity), Total Revenue (SUM Revenue), Total Profit (SUM Profit).**
e) **Add 3 interactive slicers with titles, white borders/shadows, dropdown style: Year (from Date), Month (from Date), Shop Name for dynamic filtering across visuals. Ensure slicers filter all visuals/KPIs correctly; build visuals with white data labels, borders, shadows, black axes: Line/Clustered Column (Revenue & Quantity by Branch), Donut/Pie (Quantity by Shop Name), Column (Revenue by City), Treemap (Quantity by Product Category), Donut (Revenue by Product Category), small card (Average Quantity).**

Aim

The primary aim of the question is to build an attractive, interactive **Coffee Shop Sales Dashboard** in Power BI Desktop using the provided "Coffee Shop Sales Data" Excel sheet. It emphasizes creating a visually appealing report with a thematic background (coffee shop image), professional title, key performance indicator (KPI) cards for core sales metrics, time- and location-based slicers for filtering, and a mix of chart types to analyze revenue, quantity, and performance across branches/shops, cities, product categories, and time periods. The dashboard should demonstrate strong Power BI skills in data loading, formatting, interactivity, and thematic design, while being saved as a .pbix file and prepared for viva-style explanation of insights (e.g., top-performing shops or categories) or challenges (e.g., date handling or background fitting).

Steps to Solve the Question

Follow these step-by-step instructions to create the dashboard as specified. The dataset is likely a typical coffee shop sales Excel file (common on Kaggle/GitHub) with columns such as: Transaction ID, Date (transaction_date), Quantity (transaction_qty), Revenue (often calculated as unit_price × quantity), Profit (if pre-calculated or derivable), Shop Name / Store Name / Branch (store_location or store_id mapped to name), City (derived from location), Product Category (product_category), etc. If Revenue or Profit aren't direct columns, create them via DAX or Power Query using unit_price if available.

Step 1: Import and Prepare Data (Part a)

1. Open Power BI Desktop → **Get Data** → **Excel** → Select the file → Choose the sheet named **Coffee Shop Sales Data** → Load or **Transform Data**.
2. In Power Query Editor:
    - Check/fix data types:

        - Date column → Change Type → Date.
        - Quantity → Whole Number.
        - Revenue, Profit (if present) → Decimal Number or Currency.
        - Other columns (Shop Name, Product Category, City) → Text.
    - If needed, add columns:
        - Year = YEAR([Date])
        - Month = FORMAT([Date], "MMMM") or MONTH([Date]) for name/number.
    - Remove any completely blank rows or fix obvious errors (e.g., nulls in key fields).
3. Click **Close & Apply** to load the data model.

Step 2: Set Canvas Background (Part b)

1. On the report page → **Format** pane (paint roller icon) → **Canvas background**.
2. Click **Add image** or browse → Import a coffee shop-themed image.
    - Suggested sources (free for use): Search online for "cozy coffee shop interior background" on sites like:
        - Freepik.com (coffee background vectors/photos)
        - Pexels.com (free coffee shop background stock photos)
        - Unsplash.com (high-quality coffee shop/cafe images)
    - Choose a warm, inviting image (e.g., coffee cups on table, cafe interior, beans, or subtle brown tones) — avoid busy/overly detailed ones.
3. Set **Fit** to **Page** (or **Actual size** if needed) → Adjust **Transparency** to 20–40% for readability of visuals over the image → Apply for an attractive, professional look.

Step 3: Add Main Title (Part c)

1. **Insert** → **Text Box**.
2. Type: **Coffee Shop Sales Dashboard**.
3. Format:
    - Font size: 28
    - Color: White
    - Bold: Yes
    - Alignment: Center
    - Add shadow (Format → Effects → Shadow) or semi-transparent dark background rectangle behind it for contrast.
4. Position at the top center.

Step 4: Create 4 KPI Cards (Part d)

1. Create or use these measures (Modeling tab → New measure):
    - **Total Transactions** = COUNTROWS('Coffee Shop Sales Data') (or DISTINCTCOUNT([Transaction ID]))
    - **Total Quantity** = SUM([Quantity])
    - **Total Revenue** = SUM([Revenue])
    - **Total Profit** = SUM([Profit])
2. Drag each measure to a separate **Card** visual.
3. Format each Card:
    - Enable Title (e.g., "Total Transactions", "Total Revenue").
    - White borders/shadows (Format → General → Effects → Border & Shadow).
    - Colored backgrounds (e.g., soft brown/green/teal via Format → Background).
    - Currency formatting for Revenue/Profit, whole numbers for others.

4. Arrange in a row below the title.

Step 5: Add 3 Interactive Slicers (Part e)

1. Create slicers:
    - **Year**: Drag Year (from Date) → Dropdown style.
    - **Month**: Drag Month (from Date) → Dropdown.
    - **Shop Name**: Drag Shop Name (or store_location) → Dropdown.
2. For each slicer:
    - Add title (e.g., "Select Year").
    - White borders/shadows (Format → General → Effects).
    - Dropdown style for clean look.
3. Position them (e.g., left sidebar or top row) → They will automatically filter all visuals and KPIs (verify by selecting a value).

Step 6: Build Visuals (Part e)

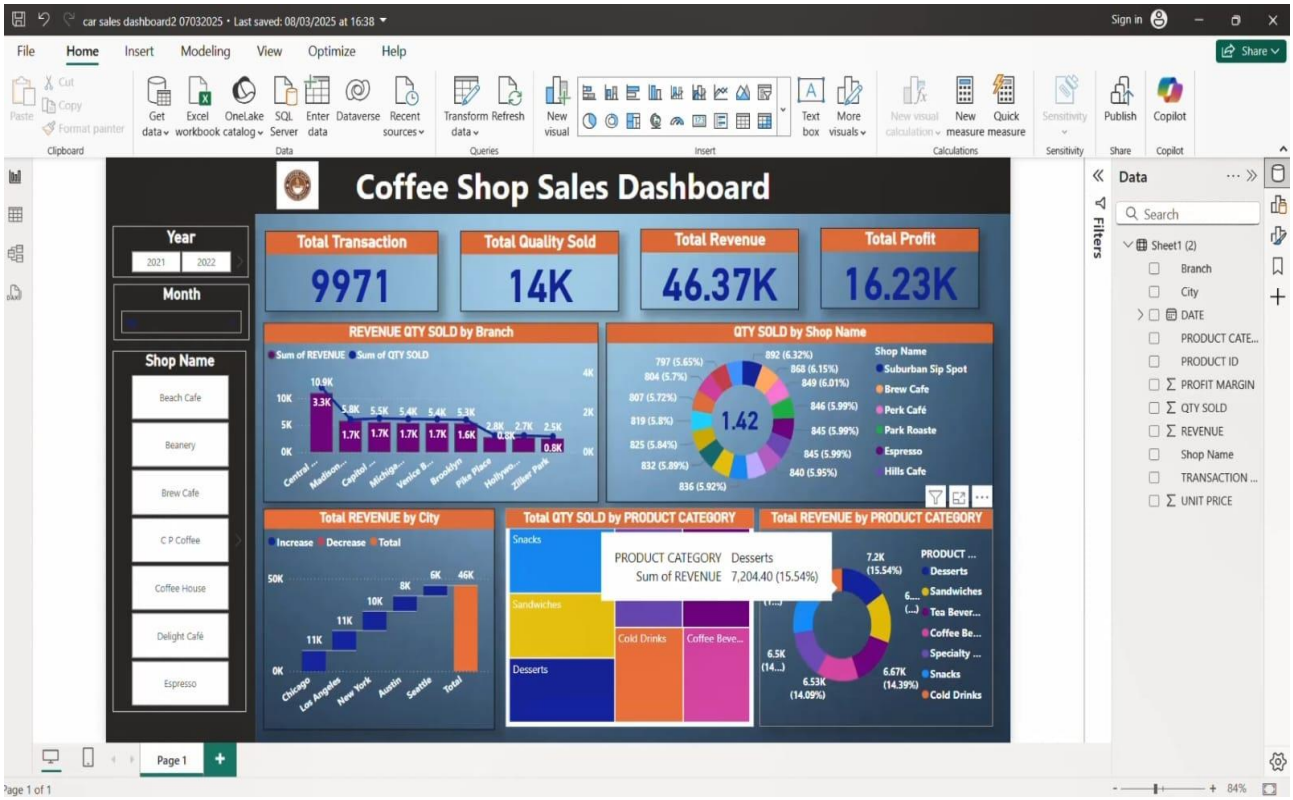
Apply consistent formatting: White data labels, borders, shadows (Format → General → Effects); black axes/labels for contrast on the background.

1. **Line/Clustered Column (Revenue & Quantity by Branch)**:
    - Clustered Column Chart or Combo Chart → Axis: Branch/Shop Name → Values: Total Revenue (column) + Total Quantity (line).
2. **Donut/Pie (Quantity by Shop Name)**:
    - Donut Chart → Legend: Shop Name → Values: Total Quantity.
3. **Column (Revenue by City)**:
    - Column Chart → Axis: City → Values: Total Revenue.
4. **Treemap (Quantity by Product Category)**:
    - Treemap → Group: Product Category → Values: Total Quantity.
5. **Donut (Revenue by Product Category)**:
    - Donut Chart → Legend: Product Category → Values: Total Revenue.
6. **Small Card (Average Quantity)**:
    - Card visual → Measure: AVERAGE([Quantity]) → Title: "Average Quantity per Transaction".
    - Format similarly (white borders/shadows, colored background).

Arrange visuals logically (e.g., KPIs top, slicers left, main charts center/bottom).

Step 7: Final Checks, Save, and Prepare for Viva

- Test: Select a Month/Year/Shop Name → Confirm KPIs, charts, and filters update (e.g., one shop shows higher revenue in a specific category).
- Ensure theme consistency (white text/labels readable on background).
- Save: File → Save As → "CoffeeShopSalesDashboard.pbix".
- **Insights examples**:
    - "Top shop/branch drives 40% of total revenue; Coffee category dominates quantity sold; Peak months show seasonal trends."
- **Challenges examples**:
    - Fitting background image without obscuring text (adjust transparency); Deriving Year/Month if not present; Ensuring slicers sync correctly with all visuals.


car sales dashboard2 07032025 • Last saved: 08/03/2025 at 16:38
File
Home
Insert
Modeling
View
Optimize
Help
Coffee Shop Sales Dashboard
Year
2021
2022
Month
Shop Name
Beach Cafe
Beanery
Brew Cafe
C P Coffee
Coffee House
Delight Café
Espresso
Total Transaction
9971
Total Quality Sold
14K
Total Revenue
46.37K
Total Profit
16.23K
REVENUE QTY SOLD by Branch
Sum of REVENUE
Sum of QTY SOLD
QTY SOLD by Shop Name
Shop Name
Suburban Sip Spot
Brew Cafe
Perk Café
Park Roaste
Espresso
Hills Cafe
1.42
Total REVENUE by City
Increase
Decrease
Total
Total QTY SOLD by PRODUCT CATEGORY
Snacks
Sandwiches
Desserts
Cold Drinks
Coffee Beve...
PRODUCT CATEGORY Desserts
Sum of REVENUE 7,204.40 (15.54%)
Total REVENUE by PRODUCT CATEGORY
PRODUCT ...
Desserts
Sandwiches
Tea Bever...
Coffee Be...
Specialty ...
Snacks
Cold Drinks
Data
Search
Sheet1 (2)
Branch
City
DATE
PRODUCT CATE...
PRODUCT ID
PROFIT MARGIN
QTY SOLD
REVENUE
Shop Name
TRANSACTION ...
UNIT PRICE
Filters
Page 1
Page 1 of 1
84%

5. Using the provided Production Working Hours dataset create an interactive Production Hours Analysis Dashboard in Power BI Desktop with these requirements .

a) **Import Excel data sheet; verify no nulls/empty rows in Power Query, create Weekday column as FORMAT(Starting Date, 'DDDD'), then Close & Apply.**
b) **Add main title textbox 'Production Working Hours Analysis Dashboard' (font size 24, bold, black/white text, centered with shadow or background).**
c) **Create 5 formatted KPI Cards with titles enabled, borders/shadows, custom colors: Total Employees (COUNT Employee Name), Total Working Hours (SUM Working Hours), Non-Working Hours (SUM Non-Working Hours), Planning Hours (SUM Planning Hours), Overtime Hours (SUM Overtime Hours).**
d) **Add 5 interactive slicers with titles, borders, shadows, dropdown style, no headers: Month (from Starting Date), Year (from Starting Date), Employee Name, Job Title, Gender for dynamic filtering.**
e) **Build visuals with data labels, borders, shadows, consistent theme: Donut Chart (Employees by Gender), Clustered Column (Amount Learned by City), Donut (Working Hours by Weekday), Clustered Column (Working Hours & Planning Hours by Division), Table (Job Title, Employee Name, Working Hours, Planning Hours, Non-Working Hours, Overtime Hours, Starting/Ending Date with conditional data bars on hours columns).**

Aim

The main objective of this question is to develop an interactive Production Working Hours Analysis Dashboard in Power BI Desktop using the provided Production Working Hours dataset (typically an Excel file containing employee-level production data). It tests end-to-end Power BI proficiency: data cleaning and enhancement in Power Query, professional layout design with title and themed elements, KPI cards for workforce utilization metrics, multi-dimensional slicers for filtering by time, employee, role, and demographics, and a variety of visuals to analyze working patterns, gender distribution, performance by city/division/weekday, and detailed breakdowns. The dashboard should highlight insights into productivity (e.g., overtime vs. planning hours, weekday efficiency), support dynamic exploration, be saved as a .pbix file, and be ready for viva discussion on insights (e.g., gender balance, division-wise hours) or challenges (e.g., handling dates, conditional formatting).

Steps to Solve the Question

Follow these step-by-step instructions to build the dashboard precisely as required. The dataset (often from sources like colorstech.net or similar tutorials) typically includes columns such as: Employee Name, Job Title, City, Gender, Division, Starting Date, Ending Date, Working Hours, Planning Hours, Non-Working Hours, Overtime Hours, Rate (hourly pay), Amount Earned/Budget, etc.

Step 1: Import and Prepare Data in Power Query (Part a)

1. Open Power BI Desktop → **Get Data** → **Excel** → Select the file → Choose the relevant sheet → **Transform Data**.
2. In Power Query Editor:

- Verify no nulls/empty rows: Go to **Home** → **Remove Rows** → **Remove Blank Rows**; check key columns (e.g., Employee Name, Starting Date, Working Hours) for nulls → Replace nulls with 0 for numeric fields (e.g., hours) or appropriate text if needed.
- Create Weekday column: Select **Starting Date** → **Add Column** → **Date** → **Day** → **Name of Day** (or use Custom Column with formula: Date.DayOfWeekName([Starting Date]) for full name like "Monday").
  - Ensure format is 'DDDD' equivalent (full weekday name): Use **Format** → **Date** → **Day Name** if available.
- Fix data types if needed: Starting Date & Ending Date → Date; Hours columns & Overtime → Decimal Number or Whole Number; Gender, Job Title, City, Division → Text.
- Optional: Add Year and Month columns for slicers → **Add Column** → **Date** → **Year** from Starting Date; **Month** → **Month Name**.

3. Click **Close & Apply** to load.

Step 2: Add Main Title (Part b)

1. On the report page: **Insert** → **Text Box**.
2. Enter: **Production Working Hours Analysis Dashboard**.
3. Format:
   - Font size: 24
   - Bold: Yes
   - Text color: Black or White (choose based on theme/background contrast)
   - Center align
   - Add shadow (Format pane → Effects → Shadow) or apply a semi-transparent background rectangle (Insert → Shapes → Rectangle behind text).
4. Position at the top center.

Step 3: Create 5 KPI Cards (Part c)

1. Create these measures (Modeling tab → New measure):
   - **Total Employees** = DISTINCTCOUNT('Table'[Employee Name]) (or COUNTROWS(SUMMARIZE(Table, [Employee Name])))
   - **Total Working Hours** = SUM('Table'[Working Hours])
   - **Non-Working Hours** = SUM('Table'[Non-Working Hours])
   - **Planning Hours** = SUM('Table'[Planning Hours])
   - **Overtime Hours** = SUM('Table'[Overtime Hours])
2. Drag each measure to a separate **Card** visual.
3. Format each:
   - Enable Title (e.g., "Total Employees", "Total Working Hours").
   - Add borders & shadows (Format → General → Effects).
   - Custom colors: Use Format → Background/Data colors (e.g., blue for Working, red for Overtime, green for Planning).
   - Format values: Whole number or 2 decimals for hours.
4. Arrange in a row below the title.

Step 4: Add 5 Interactive Slicers (Part d)

1. Create slicers:
   - **Month**: Drag Month (from Starting Date) → Dropdown style.
   - **Year**: Drag Year (from Starting Date) → Dropdown.

    - **Employee Name**: Drag Employee Name → Dropdown.
    - **Job Title**: Drag Job Title → Dropdown.
    - **Gender**: Drag Gender → Dropdown.
2. For each slicer:
    - Add title (e.g., "Select Month").
    - Borders & shadows (Format → General → Effects).
    - Dropdown style; hide headers if interpreted as "no headers" (Format → Slicer header → Off).
3. Position (e.g., left sidebar) → They filter all visuals/KPIs by default.

Step 5: Build Visuals (Part e)

Apply a consistent professional theme (View → Themes → e.g., "Default" or dark/light corporate). Enable data labels, borders, shadows on all visuals (Format → General → Effects).

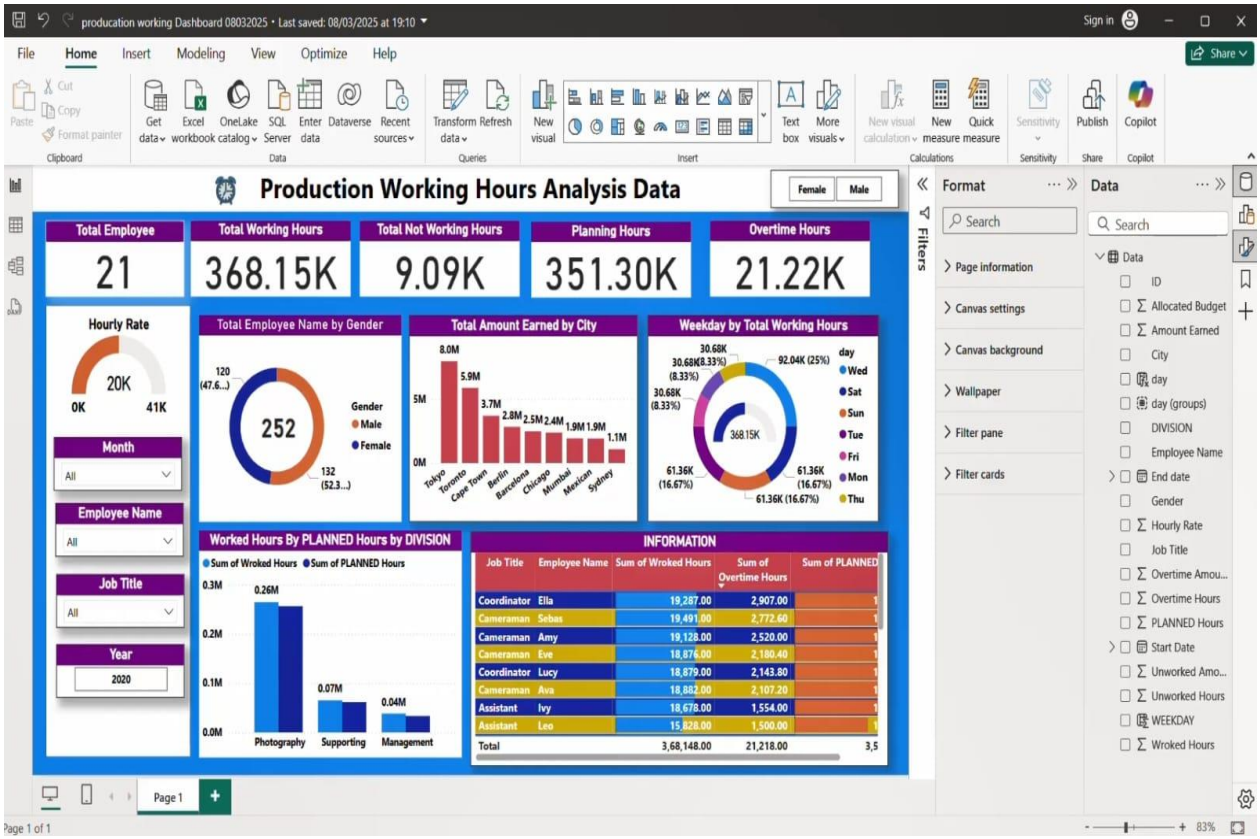
1. **Donut Chart – Employees by Gender**:
    - Donut → Legend: Gender → Values: Total Employees (measure).
2. **Clustered Column – Amount Learned by City**:
    - Likely typo/misnomer; interpret as "Amount Earned" or "Budget/Amount" by City (common in such datasets).
    - Clustered Column → Axis: City → Values: SUM(Amount Earned) or related budget column.
3. **Donut – Working Hours by Weekday**:
    - Donut → Legend: Weekday → Values: Total Working Hours.
4. **Clustered Column – Working Hours & Planning Hours by Division**:
    - Clustered Column → Axis: Division → Values: Total Working Hours + Planning Hours (two series).
5. **Table – Detailed Breakdown**:
    - Table or Matrix visual.
    - Columns/Fields: Job Title, Employee Name, Working Hours, Planning Hours, Non-Working Hours, Overtime Hours, Starting Date, Ending Date.
    - Add conditional formatting: Select hours columns → Format → Conditional formatting → Data bars (e.g., green for higher values).
    - Enable totals if needed.

Arrange visuals: KPIs top, slicers left, donuts/columns center, table bottom for details.

Step 6: Final Checks, Save, and Prepare for Viva

- Test: Filter by Gender (e.g., "Male"), Month, or Job Title → Verify KPIs update and visuals reflect (e.g., higher overtime in certain divisions).
- Ensure cross-filtering and theme consistency.
- Save: File → Save As → "ProductionHoursAnalysisDashboard.pbix".
- **Insights examples**:
    - "Significant overtime in specific divisions/cities; weekdays like Monday/Friday show lower planning adherence; gender distribution may reveal imbalances in certain job titles."
- **Challenges examples**:

- Ensuring Weekday column uses correct full name format; handling potential duplicates in Employee Name; applying data bars effectively in table for quick visual comparison.



6. Using the provided E-commerce Sales dataset create an interactive E-commerce Sales Dashboard in Power BI Desktop with these requirements.

a) **Import E-commerce Excel and State logistics CSV; in Model view, relate State columns (one-to-many), create Calendar table via DAX with Year/Month/Day, relate to Order Date, then apply changes.**
b) **Add main title textbox 'E-commerce Sales Dashboard' at top (font size 24, bold, white text, centered with shadow or background for emphasis).**
c) **Create 5 formatted KPI Cards with titles enabled, white text, borders/shadows, custom backgrounds: Total Customers (DISTINCTCOUNT Customer ID), Total Sales (SUM Sales per Order), Total Profit (SUM Profit), Total Quantity (SUM Order Quantity), Total States (DISTINCTCOUNT State).**
d) **Add 4 interactive slicers with titles, white borders/shadows, dropdown style mostly: State, Product Name, Delivery Status (no title), Customer Segment for dynamic cross-filtering of visuals and cards.**
e) **Build visuals with white data labels, borders, shadows, consistent dark/white theme: Table (Category with YTD Sales, PYTD Sales, Year over Year, Sales Icon via DAX), Bar Chart (Top 5 Products by Total Sales, Top N filter), Map Visual (Sales by State using lat/long bubbles, Region legend), Donut Chart (Sales by Shipment Type), Bar Chart (Bottom 5 Products by Total Sales, Bottom N filter).**

Aim :

The core aim of this question is to construct a comprehensive, interactive E-commerce Sales Dashboard in Power BI Desktop by integrating two data sources: an main E-commerce Sales Excel file (containing transactional data like orders, customers, products, sales, profit, etc.) and a supplementary State logistics CSV (likely containing state-level details such as State name, possibly lat/long coordinates for mapping, logistics info, or region groupings). The focus is on data modeling (relationships and a DAX Calendar table for time intelligence), professional UI elements (title, themed KPIs and slicers), and advanced visuals including time-based comparisons (YTD vs. PYTD, YoY), geographic mapping, product performance rankings (top/bottom), and shipment analysis. This demonstrates skills in multi-source data integration, DAX for calculations/measures/icons, geographic visualization, dynamic filtering, and dark-themed reporting. The dashboard must be saved as a .pbix file and prepared for viva explanation of insights (e.g., regional sales patterns, top products, YoY growth) or challenges (e.g., relationship setup, handling logistics data, creating conditional icons).

Steps to Solve the Question

Follow these step-by-step instructions to build the dashboard as specified. Assume the E-commerce Excel has columns like: Order Date, Customer ID, Customer Segment, Product Name, Category, Sales (or Sales per Order), Profit, Order Quantity, State, Shipment Type (or Ship Mode), Delivery Status, etc. The State logistics CSV likely has State, possibly Latitude, Longitude (for map bubbles), Region, or logistics metrics.

Step 1: Import Data, Model Relationships, and Create Calendar Table (Part a)

1. Open Power BI Desktop → **Get Data** → **Excel** → Import the E-commerce sales file (select relevant sheet/table).
2. **Get Data** → **Text/CSV** → Import the **State logistics CSV**.

3. In **Model** view (left sidebar icon):
    - Relate the two tables: Drag **State** from E-commerce table (many side) to **State** in State logistics table (one side) → Set relationship as **One-to-Many** (State logistics → E-commerce), single direction cross-filter.
    - Ensure cardinality is correct (1:*); activate if needed.
4. Create Calendar table via DAX (Modeling tab → New table):

    text

    ```
    Calendar =
    VAR MinDate = MIN('E-commerce'[Order Date])
    VAR MaxDate = MAX('E-commerce'[Order Date])
    RETURN
    ADDCOLUMNS(
        CALENDAR(MinDate, MaxDate),
        "Year", YEAR([Date]),
        "Month", FORMAT([Date], "MMMM"),
        "Month Number", MONTH([Date]),
        "Day", DAY([Date]),
        "Year-Month", FORMAT([Date], "YYYY-MM"),
        "Quarter", "Q" & FORMAT([Date], "Q")
    )
    ```

    - Mark as Date table: Right-click Calendar table → Mark as date table → Select [Date] column.
5. Relate Calendar[Date] to E-commerce[Order Date] (Many-to-One, single direction from Calendar to fact).
6. Click **Apply changes** (if prompted) or just proceed.

Step 2: Add Main Title (Part b)

1. On the report page: **Insert** → **Text Box**.
2. Type: **E-commerce Sales Dashboard**.
3. Format:
    - Font size: 24
    - Bold: Yes
    - Text color: White
    - Center align
    - Add shadow (Format pane → Effects → Shadow) or dark semi-transparent background for emphasis.
4. Position at the top center.

Step 3: Create 5 KPI Cards (Part c)

1. Create these measures (Modeling → New measure):
    - **Total Customers** = DISTINCTCOUNT('E-commerce'[Customer ID])

- **Total Sales** = SUM('E-commerce'[Sales per Order]) (or SUM(Sales) if column named Sales)
- **Total Profit** = SUM('E-commerce'[Profit])
- **Total Quantity** = SUM('E-commerce'[Order Quantity])
- **Total States** = DISTINCTCOUNT('E-commerce'[State])

2. Drag each to a **Card** visual.
3. Format each Card:
   - Enable Title (e.g., "Total Customers").
   - White text (Data label color).
   - Borders & shadows (Format → General → Effects).
   - Custom backgrounds (e.g., dark blue/green/teal via Background).
   - Currency formatting for Sales/Profit, whole numbers for others.
4. Arrange in a row below the title.

Step 4: Add 4 Interactive Slicers (Part d)

1. Create slicers:
   - **State**: Drag State → Dropdown style.
   - **Product Name**: Drag Product Name → Dropdown (or search-enabled).
   - **Delivery Status**: Drag Delivery Status → List or Dropdown (no title: Format → Slicer header → Off).
   - **Customer Segment**: Drag Customer Segment → Dropdown.
2. For each (except Delivery Status title):
   - Add title (e.g., "Select State").
   - White borders & shadows (Format → General → Effects).
   - Dropdown style mostly.
3. Position (e.g., left sidebar or top) → They cross-filter all visuals and KPIs.

Step 5: Build Visuals (Part e)

Apply a consistent dark/white theme: **View** → **Themes** → Choose dark theme (or custom: dark background, white text/labels). Enable white data labels, borders, shadows on all visuals.

1. **Table – Category with YTD Sales, PYTD Sales, YoY, Sales Icon**:
   - Matrix or Table visual.
   - Rows: Category
   - Values: Create DAX measures:
     - **YTD Sales** = TOTALYTD([Total Sales], 'Calendar'[Date])
     - **PYTD Sales** = CALCULATE([YTD Sales], SAMEPERIODLASTYEAR('Calendar'[Date]))
     - **Year over Year** = DIVIDE([YTD Sales] - [PYTD Sales], [PYTD Sales])
     - **Sales Icon** = SWITCH(TRUE(), [Year over Year] > 0.1, "↑ Strong Growth", [Year over Year] > 0, "↑ Growth", [Year over Year] < -0.1, "↓ Decline", "→ Stable")
   - Add conditional formatting on YoY (e.g., icons or colors); show Sales Icon as text or conditional icon set.
2. **Bar Chart – Top 5 Products by Total Sales**:
   - Bar/Column Chart → Axis: Product Name → Values: Total Sales
   - Filters pane: Top N → Top 5 by Total Sales (show items).
3. **Map Visual – Sales by State**:

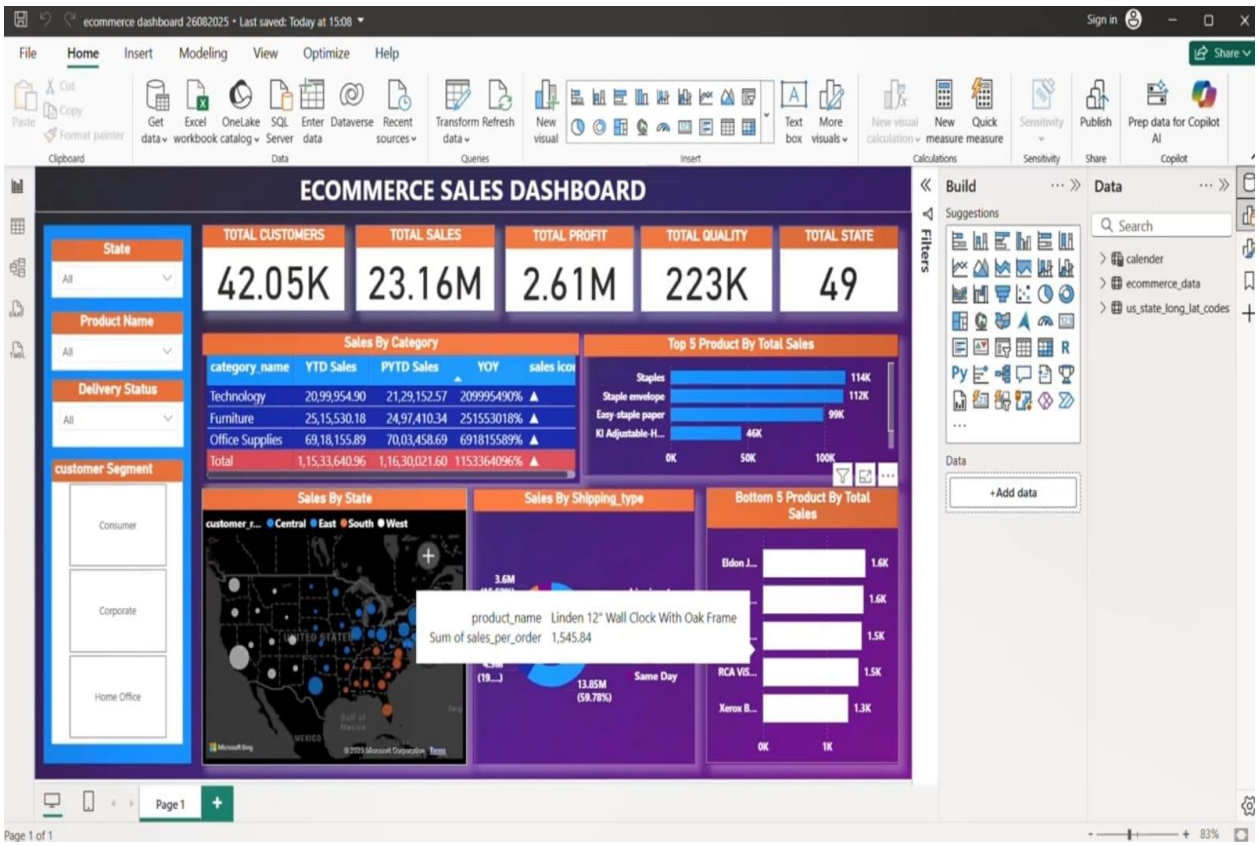
- Map (or Filled Map/Bubble Map) → Location: State (or use lat/long from State logistics if present: Latitude → Latitude, Longitude → Longitude)
- Size: Total Sales
- Legend/Color: Region (if available in logistics CSV) or State
- Use bubbles for proportional sizing.

4. **Donut Chart – Sales by Shipment Type**:
   - Donut → Legend: Shipment Type → Values: Total Sales
5. **Bar Chart – Bottom 5 Products by Total Sales**:
   - Similar to Top 5 → Filters pane: Bottom N → Bottom 5 by Total Sales.

Arrange: KPIs top, slicers left, map prominent center, table and charts around.

Step 6: Final Checks, Save, and Prepare for Viva

- Test: Filter by State or Segment → Confirm map zooms, KPIs update, YoY changes.
- Ensure relationships enable proper time intelligence and geography.
- Save: File → Save As → "EcommerceSalesDashboard.pbix".
- **Insights examples**:
  - "Top products drive majority of sales; certain states show high bubble size indicating logistics importance; positive YoY in key categories."
- **Challenges examples**:
  - Correct one-to-many relationship to avoid duplication; Accurate Calendar relation for YTD/PYTD; Sourcing/matching lat/long for map bubbles.



7. "Using the provided Zepto E-commerce dataset ,create an interactive Zepto Analysis Dashboard in Power BI Desktop with these requirements.

a) **Import Excel data into Power BI; in Power Query, handle null values, remove extra columns, create calculated column for Total Revenue as Discounted Selling Price * Quantity, then Close & Apply.**
b) **Add main title textbox 'Zepto Analysis Dashboard' at top (font size 28, white text, bold, centered, with white border and shadow effect).**
c) **Create 5 formatted KPI Cards with titles enabled, white text/borders/shadows, custom backgrounds: Total Revenue (Sum), Total Products Sold (Sum Quantity), Average Discount Price (Average), Average Selling Price (Average), Number of Out-of-Stock Items (DAX measure: CALCULATE(COUNTROWS(Table), OutOfStock = TRUE)).**
d) **Add 2 interactive slicers with titles, white borders/shadows, dropdown style, header disabled: Category and Product Name for dynamic filtering across all visuals and cards.**
e) **Build visuals with white data labels, white borders/shadows, consistent dark theme: Column Chart (Total Quantity by Category), Donut Chart (Number of Out-of-Stock Items by Category), Area Chart (MRP by Discounted Selling Price), Column Chart (Total Revenue by Category), Bar Chart (Top 10 Products by Quantity using Top N filter).**

ZEPTO Analysis Using SQL Server

1) **Find the Total Revenue Generated. 2) Find the Total Discount Given.3) Find the Average Discount % for Snacks Category.4) Find the Category with Highest Revenue.5) Find the Top 3 Products by Discount %.6) Find the Product with Highest Total Revenue.7) Find the Average MRP per Category.8) Find the Total Quantity Sold per Category.**

Aim of the Question

The primary aim of the question is to perform a comprehensive analysis of the Zepto E-commerce dataset using two complementary tools: Power BI Desktop for creating an interactive dashboard to visualize sales performance, revenue, discounts, quantity sold, and stock status with dynamic filtering, and SQL Server for executing targeted queries to extract aggregated insights such as total revenue, discounts, category-wise averages, top products, and quantity distributions. This dual approach demonstrates skills in data visualization, interactive reporting, and SQL-based data querying, enabling users to derive actionable business insights (e.g., identifying high-revenue categories, discount impacts, or out-of-stock issues) from e-commerce data.

Steps to Solve the Question

The solution is divided into two parts: (1) Building the Power BI Dashboard, and (2) Writing SQL Queries for Analysis. Assume the dataset is an Excel file named "ZeptoEcommerce.xlsx" (or similar) with columns like Category, Product Name, Quantity, MRP (Maximum Retail Price), Discounted Selling Price, OutOfStock (TRUE/FALSE or 1/0). If needed, derive Discount = MRP - Discounted Selling Price, Discount % = (Discount / MRP) * 100. For SQL, assume the data is loaded into a SQL Server table named [ZeptoSales].

Step 1: Import and Transform Data (Part a)

1. Open Power BI Desktop → **Get Data** → **Excel** → Select "ZeptoEcommerce.xlsx" → Choose the relevant sheet → **Transform Data** to open Power Query Editor.
2. Handle null values: Select columns (e.g., Quantity, Discounted Selling Price) → **Replace Values** → Replace nulls with 0 for numerics or "Unknown" for text.
3. Remove extra columns: Identify and remove redundant ones (e.g., any unnamed or duplicate columns) → Select column → Right-click → Remove.
4. Create calculated column for Total Revenue: **Add Column** → **Custom Column** → Formula: [Discounted Selling Price] * [Quantity] → Name it "Total Revenue" → Change Type to Decimal Number or Currency.
5. Fix data types: Quantity → Whole Number; MRP, Discounted Selling Price → Decimal Number; OutOfStock → Boolean; Category, Product Name → Text.
6. Click **Close & Apply** to load.

Step 2: Add Main Title (Part b)

1. **Insert** → **Text Box**.
2. Type: **Zepto Analysis Dashboard**.
3. Format: Font size 28, white text, bold, centered.
4. Add white border and shadow: Format pane → Border (white color) → Effects → Shadow (on).
5. Position at the top center.

Step 3: Create 5 KPI Cards (Part c)

1. Create measures (Modeling → New measure):
    - **Total Revenue** = SUM('Zepto'[Total Revenue])
    - **Total Products Sold** = SUM('Zepto'[Quantity])
    - **Average Discount Price** = AVERAGE('Zepto'[Discounted Selling Price])
    - **Average Selling Price** = AVERAGE('Zepto'[Discounted Selling Price]) (or MRP if intended as original price)
    - **Number of Out-of-Stock Items** = CALCULATE(COUNTROWS('Zepto'), 'Zepto'[OutOfStock] = TRUE)
2. Drag each measure to a **Card** visual.
3. Format: Enable titles (e.g., "Total Revenue"); white text/borders/shadows (Format → Data label color white, General → Effects); custom backgrounds (e.g., dark gray/blue via Background).
4. Arrange in a row below the title.

Step 4: Add 2 Interactive Slicers (Part d)

1. Create slicers:
    - **Category**: Drag Category → Dropdown style.
    - **Product Name**: Drag Product Name → Dropdown style.
2. Format: Add titles (e.g., "Select Category"); white borders/shadows (Format → General → Effects); disable header (Format → Slicer header → Off).
3. Position on the left or top → They filter all visuals and KPIs dynamically.

Step 5: Build Visuals (Part e)

Apply a consistent dark theme: **View** → **Themes** → Select dark theme. Enable white data labels, white borders/shadows on all visuals (Format → Data labels → Color white; General → Effects).

1. **Column Chart – Total Quantity by Category**: Column Chart → Axis: Category → Values: Total Products Sold.
2. **Donut Chart – Number of Out-of-Stock Items by Category**: Donut → Legend: Category → Values: Number of Out-of-Stock Items.
3. **Area Chart – MRP by Discounted Selling Price**: Area Chart → Axis: Product Name or Category → Values: SUM(MRP) + SUM(Discounted Selling Price) (two lines/areas for comparison).
4. **Column Chart – Total Revenue by Category**: Column Chart → Axis: Category → Values: Total Revenue.
5. **Bar Chart – Top 10 Products by Quantity**: Bar Chart → Axis: Product Name → Values: Total Products Sold → Filters pane: Top N → Top 10 by Total Products Sold.

Step 6: Finalize and Save

- Test: Select a Category (e.g., "Snacks") → Verify KPIs and visuals update.
- Apply dark theme consistency; ensure interactions work.
- Save: File → Save As → "ZeptoAnalysisDashboard.pbix".
- Insights for viva: "Snacks category may show highest revenue; out-of-stock items highlight supply issues; top products drive bulk quantity."

Part 2: Zepto Analysis Using SQL Server (Queries 1-8)

Assume the data is in a SQL Server table named [ZeptoSales] with columns: Category, ProductName, Quantity, MRP, DiscountedSellingPrice, OutOfStock. Derive TotalRevenue = DiscountedSellingPrice * Quantity, Discount = MRP - DiscountedSellingPrice, DiscountPercent = (Discount / MRP) * 100. Execute these queries in SQL Server Management Studio (SSMS) after importing the Excel data via Import Wizard or SSIS.

1. **Find the Total Revenue Generated**:

   text

   ```
   SELECT SUM(DiscountedSellingPrice * Quantity) AS TotalRevenue
   FROM ZeptoSales;
   ```

2. **Find the Total Discount Given**:

   text

   ```
   SELECT SUM((MRP - DiscountedSellingPrice) * Quantity) AS
   TotalDiscount
   FROM ZeptoSales;
   ```

3. **Find the Average Discount % for Snacks Category**:

   text

```
SELECT AVG(((MRP - DiscountedSellingPrice) / MRP) * 100) AS
AvgDiscountPercent
FROM ZeptoSales
WHERE Category = 'Snacks';
```

4. **Find the Category with Highest Revenue**:

   text

```
SELECT TOP 1 Category, SUM(DiscountedSellingPrice * Quantity) AS
TotalRevenue
FROM ZeptoSales
GROUP BY Category
ORDER BY TotalRevenue DESC;
```

5. **Find the Top 3 Products by Discount %**:

   text

```
SELECT TOP 3 ProductName, AVG(((MRP - DiscountedSellingPrice) / MRP)
* 100) AS DiscountPercent
FROM ZeptoSales
GROUP BY ProductName
ORDER BY DiscountPercent DESC;
```

6. **Find the Product with Highest Total Revenue**:

   text

```
SELECT TOP 1 ProductName, SUM(DiscountedSellingPrice * Quantity) AS
TotalRevenue
FROM ZeptoSales
GROUP BY ProductName
ORDER BY TotalRevenue DESC;
```

7. **Find the Average MRP per Category**:

   text

```
SELECT Category, AVG(MRP) AS AvgMRP
```

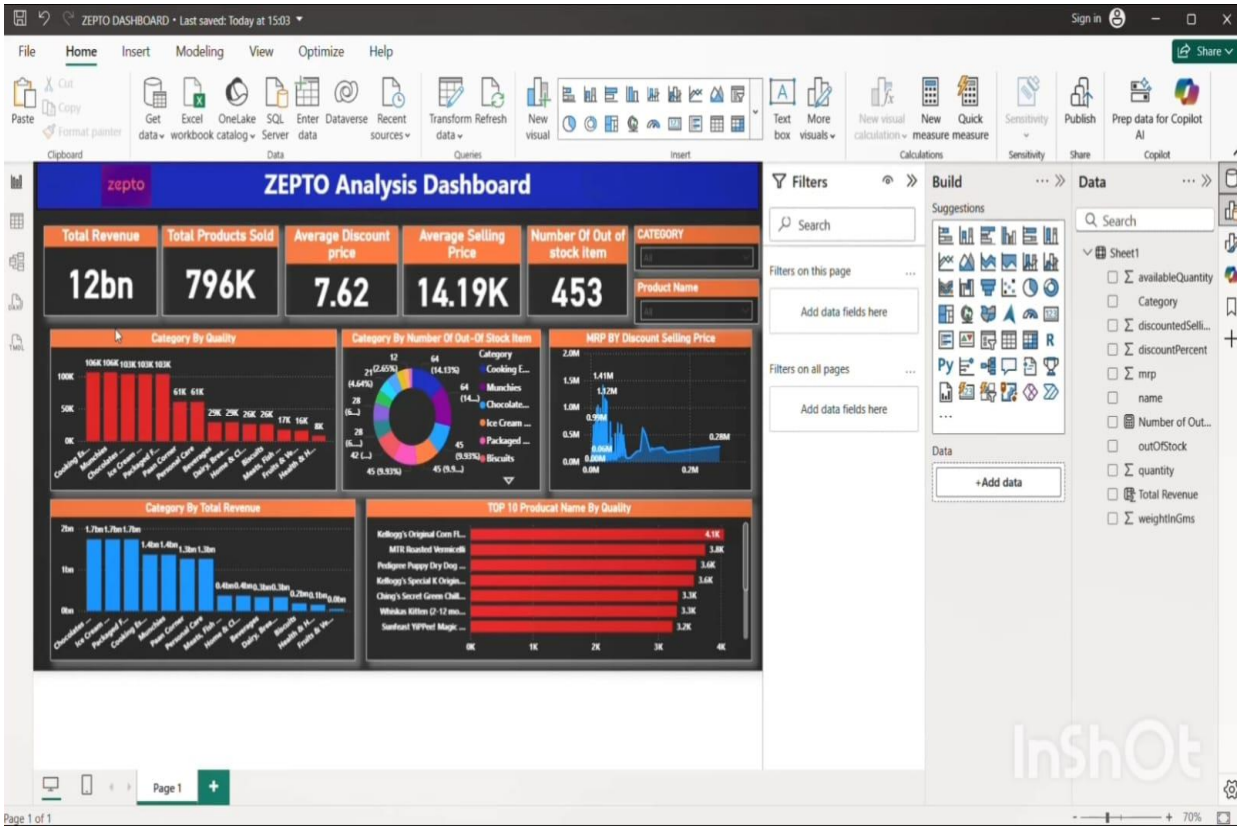
```
FROM ZeptoSales
GROUP BY Category;
```

8. **Find the Total Quantity Sold per Category**:

   text

```
SELECT Category, SUM(Quantity) AS TotalQuantitySold
FROM ZeptoSales
GROUP BY Category;
```

If the dataset file is provided (e.g., via attachment), use tools like code_execution in Python (with pandas) to verify structures or aggregates before finalizing queries. For example, to inspect: import pandas as pd; df = pd.read_excel('ZeptoEcommerce.xlsx'); print(df.head()).



8. "Using the provided sales dataset create an interactive Sales Dashboard in Power BI Desktop with these requirements.

a) **Import Excel sales data into Power BI; in Power Query, verify data types especially dates, remove any redundant columns if present, then Close & Apply to load.**
b) **Add main title textbox 'Sales Dashboard' at top (font size 20-24, bold, centered, with visible border and shadow effect for emphasis).**
c) **Create 5 formatted KPI Cards with titles enabled, borders/shadows, custom colors: Total Sales (Sum), Total Profit (Sum), Total Quantity (Sum), Total Products (DISTINCTCOUNT Product Name), Total Customers (DISTINCTCOUNT Customer ID).**
d) **Add 4 interactive slicers with titles, borders, shadows, dropdown style: City, Product Name, Month (from Order Date), Year (from Order Date) for dynamic filtering.**
e) **Build visuals with data labels, borders, shadows, consistent formatting: Pie Chart (Total Sales by Category), Pie/Donut (Total Sales by Region), Pie/Donut (Total Sales by Segment), Pie/Donut (Total Sales by Shipment Mode), Line & Clustered Column (Sales & Profit by Subcategory), Line & Clustered Column (Sales by Year), Waterfall Chart (Sales by Quarter).**

Sales Analysis Using SQL Server

1)Find the total sales and profit for each month in the year 2023.2) Which 5 cities generated the highest sales overall? 3) Identify the products where the discount is high (> 0.2) but profit is negative.4) Calculate the profit margin (Profit / Sales) for each product category.5) How many days does it take on average to ship products after order?6) Show total sales, number of orders, and average profit per segment.7) Find customers who placed more than 3 orders.8) Count how many orders were shipped using each shipping mode.9) Which region had a loss (negative profit), and how much?

Aim :

The main objective of this question is to conduct a thorough analysis of the provided sales dataset using Power BI Desktop for building an interactive dashboard that visualizes key sales metrics, performance trends, and breakdowns with dynamic filtering, and SQL Server for querying specific insights such as monthly sales/profit, top cities, high-discount products with losses, profit margins, shipping times, segment summaries, frequent customers, shipping mode counts, and regional losses. This combined approach tests proficiency in data visualization, interactive reporting, and SQL querying to uncover business insights like profitability patterns, customer behavior, regional performance, and operational efficiencies in a sales context.

Steps to Solve

The solution is split into two parts: (1) Building the Power BI Sales Dashboard, and (2) Performing Sales Analysis with SQL Queries. Assume the dataset is an Excel file (e.g., "SalesData.xlsx") with columns like Order Date, Ship Date, Sales, Profit, Quantity, Product Name, Customer ID, City, Category, Subcategory, Region, Segment, Ship Mode (or Shipment

Mode), Discount, etc. For SQL, assume the data is imported into a SQL Server table named [Sales].

Part 1: Create Interactive Sales Dashboard in Power BI Desktop (Requirements a-e)

Follow these steps to create the dashboard. The output is a professional .pbix file with interactive elements.

Step 1: Import and Transform Data (Part a)

1. Open Power BI Desktop → **Get Data** → **Excel** → Select "SalesData.xlsx" → Choose the relevant sheet → **Transform Data** to open Power Query Editor.
2. Verify/fix data types: Select columns → Change Type: Order Date & Ship Date → Date; Sales, Profit → Decimal Number or Currency; Quantity → Whole Number; Text for Product Name, Customer ID, City, etc.
3. Remove redundant columns: Identify and remove any unnecessary ones (e.g., duplicates or irrelevant IDs) → Select column → Right-click → Remove.
4. Add derived columns if needed (e.g., Month = FORMAT([Order Date], "MMMM"), Year = YEAR([Order Date]), Quarter = "Q" & FORMAT([Order Date], "Q")).
5. Click **Close & Apply** to load the model.

Step 2: Add Main Title (Part b)

1. **Insert** → **Text Box**.
2. Type: **Sales Dashboard**.
3. Format: Font size 20-24, bold, centered.
4. Add visible border and shadow: Format pane → Border (on, choose color/thickness) → Effects → Shadow (on).
5. Position at the top for emphasis.

Step 3: Create 5 KPI Cards (Part c)

1. Create measures (Modeling → New measure):
    - **Total Sales** = SUM('Sales'[Sales])
    - **Total Profit** = SUM('Sales'[Profit])
    - **Total Quantity** = SUM('Sales'[Quantity])
    - **Total Products** = DISTINCTCOUNT('Sales'[Product Name])
    - **Total Customers** = DISTINCTCOUNT('Sales'[Customer ID])
2. Drag each measure to a **Card** visual.
3. Format: Enable titles (e.g., "Total Sales"); add borders/shadows (Format → General → Effects); custom colors (e.g., blue for Sales, green for Profit via Background/Data colors).
4. Arrange in a row below the title.

Step 4: Add 4 Interactive Slicers (Part d)

1. Create slicers:
    - **City**: Drag City → Dropdown style.
    - **Product Name**: Drag Product Name → Dropdown.
    - **Month**: Drag Month (from Order Date) → Dropdown.
    - **Year**: Drag Year (from Order Date) → Dropdown.
2. Format: Add titles (e.g., "Select City"); borders/shadows (Format → General → Effects).

3. Position on the left or top → They dynamically filter all visuals and KPIs.

Step 5: Build Visuals (Part e)

Apply consistent formatting: Data labels on, borders/shadows (Format → General → Effects); uniform theme (e.g., corporate colors).

1. **Pie Chart – Total Sales by Category**: Pie → Legend: Category → Values: Total Sales.
2. **Pie/Donut – Total Sales by Region**: Donut → Legend: Region → Values: Total Sales.
3. **Pie/Donut – Total Sales by Segment**: Donut → Legend: Segment → Values: Total Sales.
4. **Pie/Donut – Total Sales by Shipment Mode**: Donut → Legend: Ship Mode → Values: Total Sales.
5. **Line & Clustered Column – Sales & Profit by Subcategory**: Combo Chart → Shared Axis: Subcategory → Column Values: Total Sales → Line Values: Total Profit.
6. **Line & Clustered Column – Sales by Year**: Combo Chart → Axis: Year → Column Values: Total Sales (or add Profit line if desired).
7. **Waterfall Chart – Sales by Quarter**: Waterfall → Category: Quarter → Breakdown: Total Sales (shows increases/decreases).

Step 6: Finalize and Save

- Test: Select a City or Month → Verify updates in KPIs and visuals.
- Ensure consistent formatting and interactions.
- Save: File → Save As → "SalesDashboard.pbix".
- Insights for viva: "Certain regions show higher sales; subcategories with low profit may need review; quarterly waterfalls highlight seasonal fluctuations."

Part 2: Sales Analysis Using SQL Server (Queries 1-9)

Assume the data is in a SQL Server table [Sales] with relevant columns. Execute these in SQL Server Management Studio (SSMS) after importing the Excel via Import Wizard.

1. **Find the total sales and profit for each month in the year 2023**:

   text

   ```
   SELECT
       FORMAT(OrderDate, 'yyyy-MM') AS Month,
       SUM(Sales) AS TotalSales,
       SUM(Profit) AS TotalProfit
   FROM Sales
   WHERE YEAR(OrderDate) = 2023
   GROUP BY FORMAT(OrderDate, 'yyyy-MM')
   ORDER BY Month;
   ```

2. **Which 5 cities generated the highest sales overall?**:

   text

```
SELECT TOP 5
    City,
    SUM(Sales) AS TotalSales
FROM Sales
GROUP BY City
ORDER BY TotalSales DESC;
```

3. **Identify the products where the discount is high (> 0.2) but profit is negative**:

   text

```
SELECT
    ProductName,
    Discount,
    Profit
FROM Sales
WHERE Discount > 0.2 AND Profit < 0;
```

4. **Calculate the profit margin (Profit / Sales) for each product category**:

   text

```
SELECT
    Category,
    SUM(Profit) / SUM(Sales) AS ProfitMargin
FROM Sales
GROUP BY Category;
```

5. **How many days does it take on average to ship products after order?**:

   text

```
SELECT
    AVG(DATEDIFF(DAY, OrderDate, ShipDate)) AS AvgShippingDays
FROM Sales;
```

6. **Show total sales, number of orders, and average profit per segment**:

   text

```
SELECT
    Segment,
    SUM(Sales) AS TotalSales,
```

```
    COUNT(DISTINCT OrderID) AS NumberOfOrders,  -- Assuming OrderID
exists; else use COUNT(*)
    AVG(Profit) AS AvgProfit
FROM Sales
GROUP BY Segment;
```

7. **Find customers who placed more than 3 orders**:

   text

```
SELECT
    CustomerID,
    COUNT(DISTINCT OrderID) AS OrderCount  -- Assuming OrderID; else
COUNT(*)
FROM Sales
GROUP BY CustomerID
HAVING COUNT(DISTINCT OrderID) > 3;
```

8. **Count how many orders were shipped using each shipping mode**:

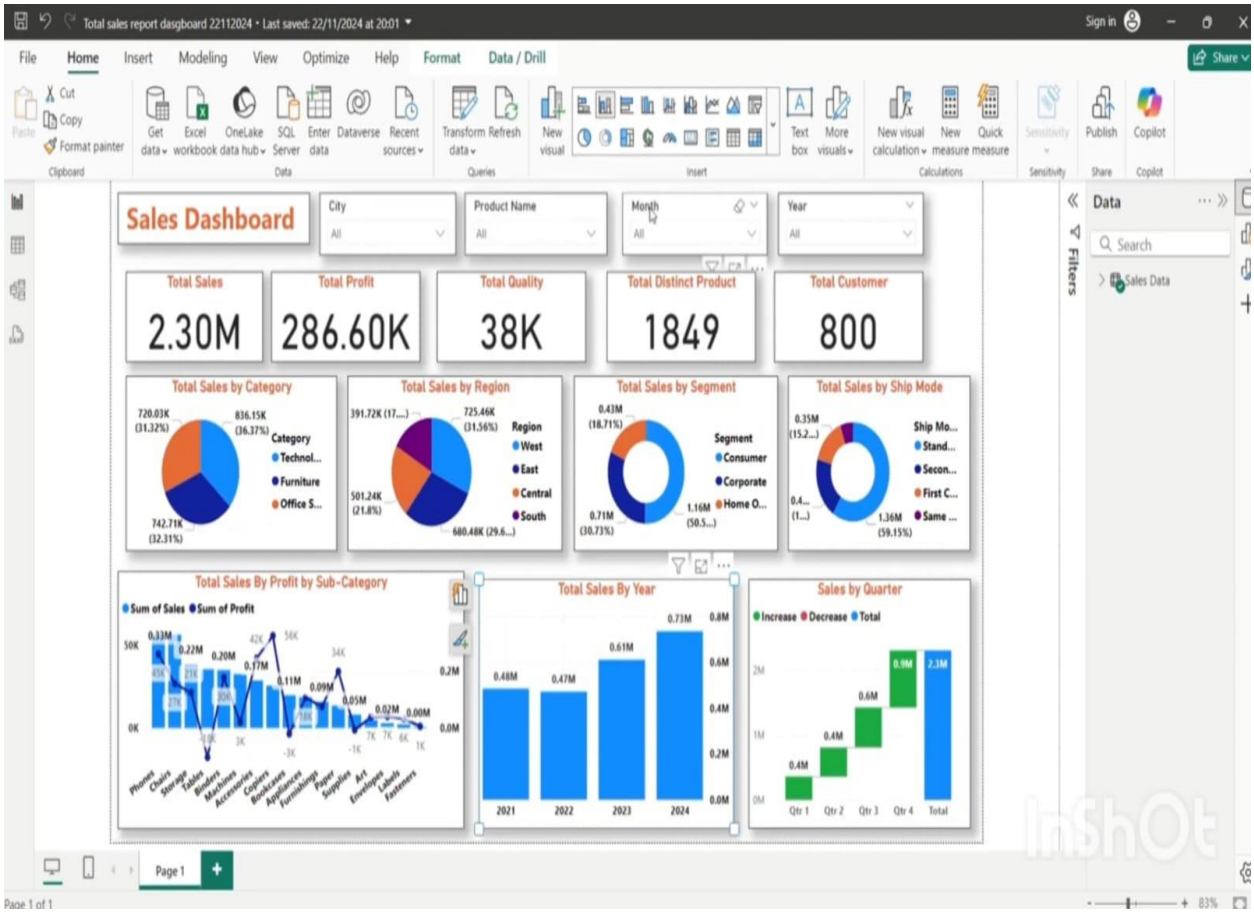
   text

```
SELECT
    ShipMode,
    COUNT(DISTINCT OrderID) AS OrderCount  -- Assuming OrderID; else
COUNT(*)
FROM Sales
GROUP BY ShipMode;
```

9. **Which region had a loss (negative profit), and how much?**:

   text

```
SELECT
    Region,
    SUM(Profit) AS TotalProfit
FROM Sales
GROUP BY Region
HAVING SUM(Profit) < 0;
```


Total sales report dasgboard 22112024 • Last saved: 22/11/2024 at 20:01
Sign in
File
Home
Insert
Modeling
View
Optimize
Help
Format
Data / Drill
Share
Cut
Copy
Format painter
Clipboard
Get data
Excel workbook
OneLake data hub
SQL Server
Enter data
Dataverse
Recent sources
Data
Transform data
Refresh
Queries
New visual
Insert
Text box
More visuals
New visual calculation
New measure
Quick measure
Calculations
Sensitivity
Publish
Share
Copilot
Sales Dashboard
City
All
Product Name
All
Month
All
Year
All
Total Sales
2.30M
Total Profit
286.60K
Total Quality
38K
Total Distinct Product
1849
Total Customer
800
Total Sales by Category
720.03K (31.32%)
836.15K (36.37%)
742.71K (32.31%)
Category
Technol...
Furniture
Office S...
Total Sales by Region
391.72K (17....)
725.46K (31.56%)
501.24K (21.8%)
680.48K (29.6...)
Region
West
East
Central
South
Total Sales by Segment
0.43M (18.71%)
0.71M (30.73%)
1.16M (50.5...)
Segment
Consumer
Corporate
Home O...
Total Sales by Ship Mode
0.35M (15.2...)
0.4... (1...)
1.36M (59.15%)
Ship Mo...
Stand...
Secon...
First C...
Same ...
Total Sales By Profit by Sub-Category
Sum of Sales
Sum of Profit
Phones
Chairs
Storage
Tables
Binders
Machines
Accessories
Copiers
Bookcases
Appliances
Furnishings
Paper
Supplies
Art
Envelopes
Labels
Fasteners
Total Sales By Year
0.48M
0.47M
0.61M
0.73M
2021
2022
2023
2024
Sales by Quarter
Increase
Decrease
Total
0.4M
0.4M
0.6M
0.9M
2.3M
Qtr 1
Qtr 2
Qtr 3
Qtr 4
Total
Page 1
Data
Search
Sales Data
Filters
InShot
Page 1 of 1
83%

9 a. Setting Up and Basic Commands - Initialize a new Git repository in a directory. Create a new file and add it to the staging area and commit the changes with an appropriate commit message.

b. Creating and Managing Branches - Create a new branch named "feature-branch." Switch to the "master" branch. Merge the "feature-branch" into "master."

Aim :

The primary aim of this question (split into parts 9a and 9b) is to assess and demonstrate fundamental Git version control skills at a beginner/intermediate level. It focuses on.

- Understanding how to set up a new local Git repository from scratch.
- Performing basic version control operations: creating files, staging changes, committing them with meaningful messages.
- Learning branching concepts: creating a new branch for isolated development (e.g., a feature), switching between branches, and merging changes back into the main branch (here called "master").

This is a classic hands-on exercise used in Git tutorials, interviews, certifications (e.g., Git basics modules), or coursework to ensure the user can:

- Initialize and work with a local repo.
- Use the core workflow of add → commit.
- Manage branches safely (create → work → merge).

It emphasizes safe, standard practices and prepares for real-world collaborative development.

Steps to Solve

Follow these exact step-by-step commands in your terminal (Git Bash, Command Prompt, PowerShell, or any shell with Git installed). Assume you are starting in an empty folder.

Part a: Setting Up and Basic Commands

```
git init

echo "Hello, this is my first file" > file1.txt

git add file1.txt

git commit -m "Initial commit: Add file1.txt with greeting message"
```

Part b: Creating and Managing Branches

```
git branch feature-branch

git checkout feature-branch

# Make changes...
```

```
echo "This is a new feature added" >> file1.txt

git add file1.txt

git commit -m "Add feature description to file1.txt"

git checkout master

git merge feature-branch
```

10. **a Creating and Managing Branches - Write the commands to stash your changes, switch branches, and then apply the stashed changes.**

b. Collaboration and Remote Repositories - Clone a remote Git repository to your local machine.

Aim :

The aim of this question (parts a and b) is to evaluate and reinforce practical knowledge of **Git's intermediate workflow features**, specifically:

- **Part a**: Handling unfinished or in-progress work when you need to switch contexts (e.g., to fix a bug on another branch or pull updates). It teaches **stashing** as a temporary shelving mechanism — saving uncommitted changes (both staged and unstaged), switching branches cleanly, and later restoring those changes. This is a very common real-world scenario in development to avoid committing half-done work or creating messy commits.
- **Part b**: Understanding how to obtain an existing project from a remote location (e.g., GitHub, GitLab, company server) to start working locally. Cloning is the standard first step for collaboration, contributing to open-source, or setting up a new development environment.

1. Make some changes in your working directory that you don’t want to commit yet

   (edit files, create new files, etc.)

2. Check the current status to confirm you have uncommitted changes

   git status

3. Stash all your current changes (both staged and unstaged)

   git stash push -m "WIP: unfinished feature X changes"

   → Your working directory and index become clean

4. Switch to the branch where you need to work now

   git checkout main

   (or `git switch main`)

5. Do your work on the target branch (e.g., fix a bug, pull updates, commit something)

   example:

git pull

make changes → git add . → git commit -m "Fix critical bug"

6. When ready, switch back to your original branch and restore the stashed changes

git checkout feature-branch

git stash pop

→ Your previous changes are now back in your working directory

Part b: Clone a remote Git repository to your local machine

1. Find/copy the repository URL you want to clone

Examples:

- HTTPS: `https://github.com/username/repo-name.git`
- SSH: `git@github.com:username/repo-name.git`

2. Open your terminal and navigate to the folder where you want the project to be saved

cd ~/projects

(or any preferred location)

3. Run the clone command with the repository URL

git clone https://github.com/username/repo-name.git

→ This creates a new folder named `repo-name` with all files and history

4. (Optional but recommended) Change into the newly cloned folder

cd repo-name

5. Verify that the clone was successful

git remote -v

→ You should see `origin` pointing to your cloned URL (fetch and push)

6. Check the default branch and recent commits to confirm everything downloaded correctly

```
git branch

git log --oneline -5
```